Lab 04 — Probabilistic Models, Built in JAX

Fundamentos de Aprendizagem Automática

Mário Antunes

2026

Overview

Class 04 · Topic 3 (second half). Lecture: `slides/04_probabilistic_models.pdf`. Demo notebooks in `notebooks/04-probabilistic-models/`: `01_mle_map.ipynb`, `02_naive_bayes_classification.ipynb` and `03_naive_bayes_regression.ipynb`.

Notebook for this lab: `notebooks/04-probabilistic-models/lab04_probabilistic_` with helpers in `lab04_utils.py` (same folder, do not edit). Cells marked **TODO** contain `raise NotImplementedError`: replace it with your code. Every other cell is ready, and it draws the plots and prints the comparisons once your functions work.

Solution: `solutions/lab04_probabilistic_models_solution.ipynb` is the fully solved notebook. Before each function it explains how the solution was reached, and after each experiment it answers this guide's questions with the numbers the run produces. Try every task yourself first.

The rule of this lab. Where a model needs an optimizer, it is *a negative log-likelihood written with* `jax.numpy` and the gradient comes from `jax.grad` (through `train`). Naive Bayes needs no gradient: its training is **counting**, and JAX scores all examples at once, in log space. Every model is compared with its scikit-learn twin and the notebook prints the largest difference.

By the end of the lab you should be able to:

1. Estimate probabilities by counting, apply Bayes' rule, derive a loss as a negative log-likelihood, add a prior, and recognize ridge and Laplace smoothing as MAP estimates.
2. Build Naive Bayes for **discrete** (Bernoulli, categorical) and **continuous** (Gaussian, exponential) features by counting, with smoothing, in log space

(floats are limited).

3. Use Naive Bayes for **regression**, with a binned target and with a continuous target, and explain where the independence assumption hurts.
4. Measure the quality of a probability (log-loss, Brier score, reliability), calibrate it, and make cost-based decisions.

Part	Task	twin	Time
A	Probability by counting and Bayes' rule; MLE and MAP with `jax.grad`: Gaussian, Beta prior, ridge as MAP	`Ridge`, `BernoulliNB` (`alpha`)	25 min
B	Naive Bayes, **discrete** features: Bernoulli, Laplace smoothing	`BernoulliNB`	15 min
C	Naive Bayes, **continuous** features: Gaussian; exponential; categorical (binned)	`GaussianNB`, `CategoricalNB`	25 min
D	Naive Bayes for **regression**: binned target; continuous target	`GaussianNB`, `CategoricalNB`, `LinearRegression`	35 min
E	Probabilities: log-loss, Brier, reliability; decisions with costs	`log_loss`, `brier_score_loss`, `calibration_curve`	20 min
F	Challenge: Platt scaling with `jax.grad`	`LogisticRegression`	home

Setup

```
cd faa
source venv/bin/activate        # see README: make venv, or pip install .
jupyter lab notebooks/04-probabilistic-models/lab04_probabilistic_models.
```

The data are those of Class 03: **spam** (Spambase, `load_spam()`) for classification and **house prices** (California housing, `load_housing()`) for regression. The notebook prepares four views of the spam features: `B_s` "the word appears" (binary), `L_s` $\log(1 + x)$ (continuous), `Z_s` four quantile bins of `L_s` (categorical), and the raw `X_s`. Scaling statistics and bin edges come from the **training** part only. `stratified_split`, `fit_standardizer`, `standardize`, `classification_metrics` and `regression_metrics` are your Class 03 functions, given ready-made.

Part A — Probability, MLE and MAP with `jax.grad`

A0. Probability by counting and Bayes' rule

A probability is a **frequency**: count and divide. Let x = "the e-mail contains `free`" and c the class. Bayes' rule turns the prior $P(c)$ and the likelihood $P(x \mid c)$ into the posterior:

$$P(c \mid x) = \frac{P(c)\,P(x \mid c)}{\sum_{c'} P(c')\,P(x \mid c')}$$

Implement `bayes_posterior(prior, likelihood)` for two arrays of shape `(K,)`. The driver estimates the prior and the likelihoods by counting on the training set, compares your posterior with the one obtained by counting *only the e-mails that contain "free"*, and then combines two words (`free` and `money`) with the **naive** assumption: the product of their likelihoods given the class.

1. Does Bayes' rule give the same number as counting directly? What is the prior, and how far does "free" move it?
2. For an e-mail with both words, compare the naive posterior with the direct count. What does the difference say about the independence assumption?

A1. Gaussian maximum likelihood

Implement `gaussian_nll(params, x)`, the negative log-likelihood of $x_1, \dots, x_n \sim \mathcal{N}(\mu, \sigma^2)$, with `params = {"mu": ..., "log_sigma": ...}`:

$$-\ell(\mu, \sigma) = \frac{n}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_i (x_i - \mu)^2$$

The driver trains it on five numbers and compares with the sample mean and with `np.var`, then measures the bias of the MLE variance by simulation.

1. Which variance does gradient descent find: the divide-by-n or the divide-by-$(n-1)$ one? Why?
2. What is the average MLE variance over many samples of size 5 from $\mathcal{N}(0, 1)$, and why is it not 1?

A2. MAP for a coin: the Beta prior

Implement `log_posterior_bernoulli(theta, k, n, a, b)` $= (k + a - 1)\log\theta + (n - k + b - 1)\log(1 - \theta)$ for a scalar θ. The driver maps it over a grid with `jax.vmap` and compares the maximum with the closed form $(k + a - 1)/(n + a + b - 2)$, then compares `BernoulliNB(alpha)` with a MAP estimate.

1. Compare MLE, Beta(2,2) MAP and Beta(10,10) MAP after 3 heads in 3 tosses, and after 30 tosses with 21 heads. What does the prior do, and when does it stop mattering?
2. Which prior does `BernoulliNB(alpha)` correspond to?

A3. MAP for regression: ridge is a Gaussian prior

Implement `map_loss(params, X, y, sigma2, tau2)`, the negative log-posterior without constants, $\frac{1}{2\sigma^2}\sum_i (y_i - w^\top x_i - b)^2 + \frac{1}{2\tau^2}\|w\|^2$ (no prior on the bias), and compare its minimizer with `Ridge(alpha = sigma2 / tau2)`.

1. How close are the two? What does σ^2/τ^2 mean as a regularization strength?

Part B — Naive Bayes with Discrete Features

$$\hat{c} = \arg\max_{c} \left[\log P(c) + \sum_{j} \log P(x_j \mid c) \right]$$

B1. Bernoulli Naive Bayes

Binary features $b_j = [x_j > 0]$. Implement `fit_bernoulli_nb(Xb, y, alpha, n_classes=2)` returning the log-prior of each class and the logs of $\theta_{cj} = (n_{cj} + \alpha)/(n_c + 2\alpha)$ and of $1 - \theta_{cj}$; `joint_log_bernoulli(model, Xb)` returning $\log P(c) + \log P(x \mid c)$ for every example and class, shape `(n, K)` (use `jnp.where(Xb == 1, log_theta, log_one_minus)`, broadcast and summed: it avoids $0 \cdot (-\infty)$); and `posterior_proba(joint)`, the normalized $P(c \mid x)$ computed with `logsumexp`.

The driver compares the joint log-probabilities and the posteriors with `BernoulliNB(alpha=1)`, prints the metrics, and counts the test e-mails that get an *impossible* class ($-\infty$) with $\alpha = 0$ and with $\alpha = 1$.

1. How close are you to scikit-learn, and how good is the classifier?
2. What does $\alpha = 0$ do with 20 training e-mails, and with 100? Why does smoothing matter most for rare words?
3. Floats are limited: the driver prints the probability of the least likely test e-mail as a product of 57 factors in `float32` and `float64`, and as a sum of logs. What fails, and why does the lab work in log space? What does smoothing add to that?

Part C — Naive Bayes with Continuous Features

C1. Gaussian Naive Bayes

Implement `fit_gaussian_nb(X, y, var_smoothing=1e-9, n_classes=2)` returning the log-prior, the class means and the class variances (the maximum-likelihood variance, **dividing by** n; add `var_smoothing * max_j Var(x_j)` to every variance), and `joint_log_gaussian(model, X)`. Broadcast the examples `(n, 1, d)` against the class parameters `(1, K, d)`: no loops over examples. Use the log features `L_s`.

1. How close are you to `GaussianNB`? Compare its metrics with those of Bernoulli NB and explain its high recall and low precision.

C2. Categorical Naive Bayes

Implement `fit_categorical_nb(Z, y, alpha, n_categories, n_classes=2)` returning the log-prior and $\log \theta_{cjm} = \log \frac{n_{cjm}+\alpha}{n_{cj}+\alpha M}$ (shape `(K, d, M)`, use `np.bincount`), and `joint_log_categorical(model, Z)`. The driver compares with `CategoricalNB(min_categories=4)` and then trains logistic regression and the three Naive Bayes models on the first n training e-mails, for growing n, and plots the test accuracy.

1. Which Naive Bayes model is best on spam? Why is the categorical one not clearly better than the Bernoulli one?
2. Which model wins with 20 examples, and which with 3680? Why do the NB curves plateau?

C3. Another distribution: the exponential

The Gaussian is not the only choice for a continuous feature. A word frequency is non-negative with a long right tail, and the **exponential** density $P(x_j \mid c) = \lambda_{cj} e^{-\lambda_{cj} x_j}$ has that shape; its maximum-likelihood rate is one over the class mean. Implement `fit_exponential_nb(X, y, n_classes=2)` returning the log-prior and the rates $\lambda_{cj} = 1/(\bar{x}_{cj} + 10^{-3})$, and `joint_log_exponential(model, X)` $= \log P(c) + \sum_j (\log \lambda_{cj} - \lambda_{cj} x_j)$. Use the **raw** frequencies. The driver checks your joint against `scipy.stats.expon`, compares four Naive Bayes models, and plots the fitted densities of one feature on its histogram.

1. Which of the four models is best on spam? How well does each distribution match the histogram?
2. The exponential's most likely value is always 0. Why can that be wrong for a word frequency, and what would match it better (look at the Bernoulli model)?

Part D — Naive Bayes for Regression

The target is now a number. Two routes.

D1. Discretize the target

Implement `bin_target(y, n_bins)` returning the interior quantile **edges** of y (`np.quantile`) and the **bin means** m_k (the class of a value is `np.searchsorted(edges,`

`y, side="right")`), and `predict_expected(joint, means)`, the expected value $\hat{y} = \sum_k P(k \mid x)\, m_k$ from the joint log-scores of a classifier (use `posterior_proba`).

The driver trains your Gaussian NB on the bin labels for $K = 5, 10, 20$, predicts the expected value and the most probable bin, and compares with `KBinsDiscretizer` + `GaussianNB`.

1. How do your bin labels and expected values compare with scikit-learn's?
2. Expected value or most probable bin? Compare their R^2 with linear regression's and explain.

D2. Discrete features for regression

Discretize the **features** as well (`fit_bins` / `apply_bins`, 3, 5, 10 or 20 quantile bins) and use your categorical NB with the K target bins.

1. How does R^2 change with the number of bins per feature? Compare with the Gaussian version and with linear regression.

D3. A continuous target: linear-Gaussian Naive Bayes

$$y \sim \mathcal{N}(\mu_y, \tau^2), \qquad x_j \mid y \sim \mathcal{N}(a_j + b_j\, y,\ s_j^2) \ \text{(independent given } y)$$

Implement `fit_lgnb(X, y)` returning $(\mu_y, \tau^2, a, b, s^2)$: the mean and variance of y, the least-squares fit of every feature **on** y (`np.linalg.lstsq` on `[y, 1]`) and the residual variances. Then `predict_lgnb(model, X)` returning the posterior mean and standard deviation of y:

$$P = \frac{1}{\tau^2} + \sum_j \frac{b_j^2}{s_j^2}, \qquad \hat{y} = \frac{1}{P}\Big(\frac{\mu_y}{\tau^2} + \sum_j \frac{b_j\,(x_j - a_j)}{s_j^2}\Big), \qquad \text{sd} = \frac{1}{\sqrt{P}}$$

The driver **verifies your formula without it**: the negative log-posterior is written in `jax.numpy`, `jax.grad` climbs it for five districts, and the maximum must equal your posterior mean. It then prints R^2 and the coverage of the 90% interval.

1. Does the closed form agree with the gradient ascent?
2. How good is the model against linear regression and against D1? What is the posterior standard deviation, and does it depend on the district?

D4. Where the naive assumption hurts

The driver appends $0, 1, 3, 10$ exact copies of the most informative feature (median income) and refits linear regression and your linear-Gaussian NB.

1. What happens to each model's R^2, to the posterior standard deviation and to the coverage of the 90% interval? Explain in terms of the independence assumption.

Part E — Probabilities and Decisions

The driver trains a logistic regression on the standardized log features to compare with your Naive Bayes models.

E1. Log-loss, Brier score and reliability

Implement `log_loss_(y, p)` $= -\frac{1}{n}\sum[y \log p + (1-y)\log(1-p)]$ (clip p to $[10^{-15}, 1-10^{-15}]$), `brier_score(y, p)` $= \frac{1}{n}\sum(p-y)^2$ and `reliability_table(y, p, bins)`, which cuts $[0,1]$ into equal bins and returns, for every **non-empty** bin, the mean predicted probability, the observed frequency of spam and the count. The driver checks them against `log_loss`, `brier_score_loss` and `calibration_curve`.

1. Compare the three models on accuracy, log-loss and Brier score. What does the log-loss of Gaussian NB say that its accuracy does not?
2. Read the reliability diagrams.

E2. Decisions with costs

A false positive (a real e-mail hidden) costs c_{FP}, a false negative (a spam delivered) costs c_{FN}. Implement `bayes_threshold(c_fp, c_fn)` $= c_{FP}/(c_{FP}+c_{FN})$ and `expected_cost(y, p, threshold, c_fp, c_fn)`, the mean cost per e-mail. The driver compares, for the costs 1:1, 10:1 and 1:5, the threshold 0.5, the Bayes threshold and the best threshold found by scanning the test set.

1. How much does the cost fall when the threshold comes from the costs? Is the theoretical threshold the best one?
2. What happens to the cost at the Bayes threshold if the probabilities come from Gaussian NB?

Part F — Challenge: Platt Scaling with `jax.grad`

Repair the probabilities of the Gaussian NB. **Platt scaling** learns a logistic map $p = \sigma(a\,t + b)$ from a transform t of the classifier's score $s = \log P(\text{spam} \mid x) - \log P(\text{ham} \mid x)$.

1. Split the training set: the first 75% fits the Gaussian NB, the last 25% is the **calibration set**.
2. The Naive Bayes log-odds reach $\pm 10^8$: compress them with $t = \text{sign}(s)\log(1 + |s|)$ (monotone). Implement `platt_loss(params, t, y)` (the binary cross-entropy of $\sigma(at+b)$, `params = {"a", "b"}`) and fit it on the calibration set with `train`.
3. On the test set report the log-loss, the Brier score, the accuracy and the cost at the Bayes threshold for 10:1 costs, before and after calibration. Compare your (a, b) with a `LogisticRegression` fitted on the same one-dimensional score.

Why must the calibration data differ from the training data? What did calibration repair, and what did it not?

Going further. Replace Platt scaling by isotonic regression (`sklearn.isotonic.IsotonicR` or use `CalibratedClassifierCV`, and compare. Or build a **multinomial** Naive Bayes for word *counts* (the natural model for text) and compare it with the Bernoulli one.